

SwarmOps Cloud

Installation and Operations Guide

Nima Sarayan

September 2026

Abstract

This guide tells a developer how to run SwarmOps Cloud locally and tells an operator how to install it, upgrade an existing SwarmOps installation to the relational database, back it up, restore it, and keep it healthy. Commands are given exactly as they are used in the repository. Where a step was not exercised in the verification environment — a production installation on Ubuntu, in particular — the guide says so.

Contents

1	Components and prerequisites	1
2	Local development	2
2.1	Database	2
2.2	Running everything	2
2.3	Development settings	2
2.4	Tests	3
3	Building	3
4	Production installation	3
4.1	Prepare the database	3
4.2	Write the DSN file	4
4.3	Install the control plane	4
4.4	Connect servers and install the gateway	4
4.5	Check readiness	4
5	Upgrading an existing SwarmOps installation	4
6	Operations	5
6.1	Background work	5
6.2	Backup	5
6.3	Restore	5
6.4	Monitoring	6
6.5	Troubleshooting	6
7	Security checklist	6

List of Figures

List of Tables

1	Components	1
2	Development settings	2
3	Troubleshooting	6

1 Components and prerequisites

Table 1: Components

Component	Purpose	Version used in verification
SwarmOps Core (<code>cmd/api</code>)	Server: APIs, storefront, console, command worker, billing	Built from the branch with Go 1.26.6
Machine agent (<code>cmd/agent</code>)	Performs Docker and Swarm operations on each node	Same source, stamped v0.22.0
MariaDB or MySQL	Controller database	MariaDB 11.4.13, MySQL 8.4.11

Component	Purpose	Version used in verification
Docker Engine in Swarm mode	Runs the gateway and customer applications	Docker 29.4.0, single node
Node.js and npm	Builds the web applications	Node.js 26.5.0
Traefik	Gateway, installed by SwarmOps	v3.6.13

The database must be MariaDB 11.4 LTS or MySQL 8.4 LTS, or a later release of the same series. SwarmOps's own container images are built for **amd64**. On an **arm64** host, run Core and the agent as native binaries, as in the verification environment.

2 Local development

2.1 Database

`make dev-db`

This starts a MariaDB 11.4 container named `swarmops-mariadb` on `127.0.0.1:3307`, with root password `devroot`. It also creates the `swarmops` database with `utf8mb4`. Running it again restarts the existing container. To test against MySQL as well, start a second server, for example on port 3308:

```
docker run -d --name swarmops-mysql -e MYSQL_ROOT_PASSWORD=devroot -p 127.0.0.1:3308:3306 my
```

2.2 Running everything

`make local`

`make local` prepares a loopback machine agent, starts Core with development authentication and connects it to the agent, and starts the Vite development server. Core listens on `127.0.0.1:8084` and the web server on `127.0.0.1:5284`. Development state lives under `$TMPDIR/swarmops-dev`; override this with `SWARMOPS_DEV_DIR`. The pieces can be run separately with `make dev-agent`, `make dev-api` and `make web-dev`.

Open the console at `http://127.0.0.1:5284/` and the storefront at `http://127.0.0.1:5284/store.html`.

2.3 Development settings

Table 2: Development settings

Variable	Meaning
<code>SWARMOPS_INSECURE_DEV_AUTH=true</code>	Enables development authentication. Never use in production.
<code>SWARMOPS_DEV_SESSION_KEY</code>	At least 32 bytes. The data-encryption key is derived from it, so keep it stable between restarts, or the database's secret columns become unreadable.
<code>SWARMOPS_DEV_PASSWORD_HASH</code>	Bcrypt hash of the development operator's password (user <code>SWARMOPS_ADMIN_USERNAME</code> , default <code>admin</code>).
<code>SWARMOPS_DEV_DATABASE_DSN</code>	Development DSN; defaults to <code>root:devroot@tcp(127.0.0.1:3307)/swarmops</code> .

Variable	Meaning
SWARMOPS_MUTATIONS_ENABLED=true	Allows changes to servers and applications, including order confirmation.
SWARMOPS_DEV_MACHINE_API_*	Connection to the loopback machine agent (set by <code>make local</code>).

Generate a password hash with, for example, `htpasswd -nbBC 10 admin 'your-password' | cut -d: -f2`.

2.4 Tests

```
go vet ./...
SWARMOPS_TEST_DATABASE_DSN='root:devroot@tcp(127.0.0.1:3307)/' SWARMOPS_TEST_DATABASE_REQUIRED=
SWARMOPS_TEST_DATABASE_DSN='root:devroot@tcp(127.0.0.1:3308)/' SWARMOPS_TEST_DATABASE_REQUIRED=
npm --prefix web run typecheck
npm --prefix web test
npm --prefix web run build
make stack-check-all TAG=ci
make docs-check
```

Each database test creates and removes its own schema, named `swarmops_test_....`. Without `SWARMOPS_TEST_DATABASE_REQUIRED`, tests that cannot reach the database are skipped rather than failed.

3 Building

```
npm --prefix web ci
npm --prefix web run build      # writes the storefront and console into internal/web/static
go build -o swarmops-core ./cmd/api
go build -ldflags "-X main.version=v0.22.0" -o swarmops-agent ./cmd/agent
```

The web build must run before `go build`, because Core embeds `internal/web/static`. Release bundles are produced by `scripts/build-release-bundles.sh`, which stamps the version.

4 Production installation

The steps in this chapter follow the repository's installer and stack definitions. They were checked by rendering the stacks and syntax-checking the scripts. They were **not** carried out on an Ubuntu server during this project (Test Plan and Report, section 8).

4.1 Prepare the database

On the database server:

```
CREATE DATABASE swarmops CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'swarmops'@'10.0.0.%' IDENTIFIED BY 'a-long-random-password';
GRANT ALL PRIVILEGES ON swarmops.* TO 'swarmops'@'10.0.0.%';
```

Core applies migrations when it starts, so the account it uses needs the right to create and alter tables. For a stricter separation, run the first start, and every upgrade, with a schema-owning account. Afterwards, restrict the everyday account to `SELECT`, `INSERT`, `UPDATE` and `DELETE`.

4.2 Write the DSN file

The DSN uses the Go MySQL driver's format:

```
swarmops:a-long-random-password@tcp(db.internal:3306)/swarmops?tls=true
```

Core always adds `parseTime=true`, UTC as its time zone (`time_zone='+00:00'`), the `utf8mb4_unicode_ci` collation and a 10-second connection timeout, so these need not be written. Store the DSN in a file that is:

- a regular file, not a symbolic link;
- readable only by its owner: no permissions for group or others, for example mode 0400 or 0600.

Core refuses to start otherwise. The same rules apply to the data-encryption key file, which holds exactly 32 bytes in standard base64.

4.3 Install the control plane

Run the bootstrap script with the DSN file:

```
sudo bash scripts/bootstrap-swarmops-control-plane.sh --database-dsn-file /root/swarmops-dat
```

The script refuses to continue without a readable, non-empty DSN file. It installs the file as `database-dsn` in Core's configuration directory, and writes `SWARMOPS_DATABASE_DSN_FILE` into Core's environment. When Core runs as a Swarm service (`deploy/stacks/swarmops.yml`), the DSN is supplied as the Swarm secret `swarmops_database_dsn`, and `scripts/run-swarmops-api.sh` requires the secret file to exist.

4.4 Connect servers and install the gateway

1. Enroll each node with the install command from **Connect a server** in the console.
2. Label the manager that will carry the gateway with `nim.edge=true`. The prerequisite repair does this for a single manager.
3. In *Traffic › Gateway settings*, set the ACME contact email and the dashboard hostname. In *Traffic › Gateway*, fix the prerequisites and install the gateway (User Manual, *Preparing the platform*).

The gateway repair requires machine agent v0.9.3 or newer. Released agents carry their version. An agent built from source without `-ldflags "-X main.version=..."` reports `dev` and is refused (Test Plan and Report, defect D-6).

4.5 Check readiness

```
curl -fsS https://core.example.com/readyz
```

`/readyz` answers `{"status":"ready"}` when the audit ledger and command storage — both in the database — are available, and 503 otherwise. Use it as the health check of any load balancer or supervisor in front of Core.

5 Upgrading an existing SwarmOps installation

Versions of SwarmOps before this project kept their state in encrypted `.sealed` files in the data directory. To move that state into the database:

1. **Back up** the data directory, including `commands/`, and the data-encryption key.
2. **Stop Core**. Two processes must not write the same state.

3. **Configure** the DSN file as in section 4.2.
4. **Import**, with the same environment Core uses (the same data directory and key):

```
swarmops-core migrate-state
```

The command creates or upgrades the schema, then imports each store in its own transaction. It prints one line per store:

```
imported core topology members          1
imported enrolled outbound agents       0
imported server profiles and retained keys 1 profiles, 1 keys
imported audit events                   1
imported applications                   0
imported database credentials            0
imported source connections              0
imported source settings                 0
imported routing clusters                0
imported commands                       0
controller database is at schema version 9; the sealed files in ... are unchanged
```

This output is from the upgrade test (Test Plan and Report, section 4.4). A store without a file imports nothing.

5. **Verify** the counts against what the old installation showed: servers, applications, audit entries and runs.
6. **Start Core**. It now reads and writes only the database.

The import leaves the `.sealed` files unchanged, and running it again creates no duplicates. To roll back, stop Core and start the previous version with its untouched data directory.

6 Operations

6.1 Background work

On the **active** controller only:

- the command worker polls for due commands every 250 ms and runs one at a time;
- billing runs every five minutes, charging elapsed hours and issuing the previous month's invoices.

A standby controller serves reads, refuses changes with 409, and does neither.

6.2 Backup

```
mysqldump --single-transaction --routines --triggers \
  --host=db.internal --user=swarmops --password swarmops > swarmops-$(date -u +%Y%m%dT%H%MZ)
```

`--single-transaction` takes a consistent snapshot of the InnoDB tables without stopping Core. Keep each backup with a separately protected copy of the data-encryption key. Secrets in the backup cannot be read without the key, and the key is not in the database. Also back up the data directory's `commands/` folder if build artifacts must survive.

6.3 Restore

1. Stop Core.
2. Create an empty database and load the dump: `mysql swarmops < swarmops-...sql`.

3. Start Core with the **same** data-encryption key. The migration runner checks the recorded checksums, applies any newer migrations and refuses a database newer than the binary.

6.4 Monitoring

- **Logs.** Core writes structured JSON logs to standard output. A failed command is logged as `command execution failed`, with its id, action, server, target, attempt and cause.
- **Runs.** *Activity* › *Runs* in the console shows every command's steps, attempts and failure summary.
- **Audit.** The audit log records who did what: every administrator decision and customer purchase event.
- **Money.** Run the ledger check periodically; it must return no rows:

```
SELECT w.user_id, w.balance_rial, COALESCE(SUM(t.amount_rial), 0) AS ledger_sum
FROM wallets w LEFT JOIN wallet_transactions t ON t.user_id = w.user_id
GROUP BY w.user_id, w.balance_rial
HAVING w.balance_rial <> COALESCE(SUM(t.amount_rial), 0);
```

6.5 Troubleshooting

Table 3: Troubleshooting

Symptom	Likely cause	Action
Core exits: ... <i>must be readable only by its owner</i>	The DSN or key file permissions are too open.	<code>chmod 0400</code> the file.
Core exits after a migration checksum error	An applied migration file was modified.	Never edit an applied migration; add a new one.
Core refuses a database <i>newer than the controller</i>	A newer version migrated this database.	Run that version, or restore a backup taken before the upgrade.
Every deployment fails with <code>gateway_required</code>	The managed gateway is not installed.	Install it (section 4.4).
A deployment stays <i>provisioning</i> for minutes, then fails	The application's health check never passes; the queue is blocked meanwhile.	Check the image answers <code>GET /healthz</code> on its port and contains <code>wget</code> or <code>curl</code> .
Order confirmation answers <i>not connected to a Swarm manager</i>	The chosen server is offline or not a manager.	Reconnect the agent; check <code>docker info</code> on the node.
Changes answer 409 <i>standby</i>	This controller is not the active one.	Use the active controller, or promote this one.
Changes answer 403 <i>Remote mutations are disabled</i>	Mutations are switched off.	Enable mutations in Core's configuration.
<code>/readyz</code> answers 503	The database is unreachable or not writable.	Check the database server and the DSN.

7 Security checklist

- DSN and key files are regular files readable only by Core's user.
- The database is reachable only from the control host, over TLS (`tls=true` in the DSN).

- `SWARMOPS_INSECURE_DEV_AUTH` is not set in production, and secure cookies are enabled (HTTPS).
- The data-encryption key is backed up separately from the database backups.
- The console is reachable only by administrators, and the storefront through HTTPS.
- Every administrator uses their own account, so the audit log names them.